# Computer Architecture Lab

## (EL-2012)

## LABORATORY REPORT

## Fall 2026




# (Lab#_10__)
# Implementation of Data Path

Student Name: _Eesha Rehan_____ Roll No: 24i-6027_____ Section: B___

Date performed: __29th November _____, 2026

**MARKS AWARDED: _____ / 10**

**Signature and date of Instructor: _____**
**Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad**

## Task 1:

**Description:**

In this task, we made a full 32-bit datapath in which the Program Counter (PC) updates the next instruction address on reset or on the positive edge of the clock. Then we made a PC Adder that adds 4 to the PC because each instruction is 4 bytes long, allowing the processor to move to the next instruction. The instruction is fetched from instruction memory, and rs1, rs2, and rd fields are sent to the register file. A PC MUX selects between normal execution and branch target address, and an Immediate Generator creates immediate values for instructions. Finally, we implemented the Register File, Comparator, ALU, and Write Back path, and integrated all modules into the datapath_32bit design.

**Code:**

```
//program counter

module pc_reg(

   input clk,

   input reset,

   input [31:0] next_pc,

   output reg [31:0] pc

);



always @(posedge clk or posedge reset)

begin

   if(reset)

      pc <= 32'h00000000;

   else

      pc <= next_pc;

end
```

```
endmodule

//pc adder

module pc_adder(

   input  [31:0] pc,

   output [31:0] pc4

);



assign pc4 = pc + 32'd4;



endmodule

//pc mux

module pc_mux(

   input [31:0] pc4,

   input [31:0] target,

   input PCSel,



   output [31:0] next_pc

);



assign next_pc = (PCSel) ? target : pc4;//0=pc4,1=target



endmodule
```

```
//instruction memory

module instruction_memory(

   input  [31:0] pc,

   output [31:0] inst

);



reg [31:0] mem [0:15];

integer i;



initial

begin

   for(i=0; i<16; i=i+1) mem[i] = 32'h00000013;

   mem[0] = 32'h00700293; // ADDI x5,x0,7

   mem[1] = 32'h00500313; // ADDI x6,x0,5

   mem[2] = 32'h006283B3; // ADD x7,x5,x6

   mem[3] = 32'h40628433; // SUB x8,x5,x6

end



assign inst = (pc[31:6] == 26'd0) ? mem[pc[5:2]] : 32'h00000013;



endmodule

//reg file

module register_file(
```

```
   input clk,
   input RegWEn,


   input [4:0] AddrA,
   input [4:0] AddrB,


   input [4:0] AddrD,
   input [31:0] DataD,


   output [31:0] DataA,
   output [31:0] DataB
);


reg [31:0] regs [0:31];
integer k; // FIX 1
initial for(k=0; k<32; k=k+1) regs[k] = 32'd0;
always @(posedge clk)
begin
   if(RegWEn && AddrD != 5'd0)
      regs[AddrD] <= DataD;
end
assign DataA =
     (AddrA == 0) ? 32'd0 : regs[AddrA];
```

```
assign DataB =
    (AddrB == 0) ? 32'd0 : regs[AddrB];
endmodule
//imm generator
module imm_generator(
  input [31:0] inst,
  input [1:0] ImmSel,
  output reg [31:0] Imm
);

always @(*)
begin
  case(ImmSel)

    2'b00:
      Imm = {{20{inst[31]}},
          inst[31:20]};

    2'b01:
      Imm = {{20{inst[31]}},
          inst[31:25],
          inst[11:7]};
```

```
    2'b10:
      Imm = {{19{inst[31]}},
          inst[31],
          inst[7],
          inst[30:25],
          inst[11:8],
          1'b0};

    default:
      Imm = 32'd0;

  endcase
end

endmodule
//branch comparartor
module branch_comparator(
  input [31:0] DataA,
  input [31:0] DataB,

  output BrEq,
  output BrLT
```

```
);



assign BrEq = (DataA == DataB);



assign BrLT = ($signed(DataA) < $signed(DataB));



endmodule

//alu

module alu(

   input [31:0] A,

   input [31:0] B,

   input [2:0] ALUSel,

   output reg [31:0] result

);



always @(*)

begin

   case(ALUSel)



      3'b000:

         result = A + B;



      3'b001:
```

```
        result = A - B;



    3'b010:

        result = A & B;



    3'b011:

        result = A | B;



    3'b100:

        result = A ^ B;



    3'b101:

        result = ($signed(A) < $signed(B))

                ? 32'd1

                : 32'd0;



    default:

        result = 32'd0;



  endcase

end



endmodule
```

```
//a mux

module a_mux(

   input [31:0] DataA,

   input [31:0] pc,

   input ASel,



   output [31:0] A

);



assign A = (ASel) ? pc : DataA;



endmodule

//b mux

module b_mux(

   input [31:0] DataB,

   input [31:0] Imm,

   input BSel,



   output [31:0] B

);



assign B = (BSel) ? Imm : DataB;
```

```
endmodule

//write back

module wb_mux(

   input [31:0] alu_result,

   input [31:0] pc4,

   input WBSel,



   output [31:0] wb

);



assign wb = (WBSel) ? pc4 : alu_result;



endmodule

//datapath

module datapath_32bit(



   input clk,

   input reset,



   input PCSel,

   input RegWEn,

   input ASel,

   input BSel,
```

```
    input [2:0] ALUSel,
    input [1:0] ImmSel,

    input WBSel,

    output [31:0] pc,
    output [31:0] inst,
    output [31:0] alu_result,
    output [31:0] wb,

    output BrEq,
    output BrLT

);
wire [31:0] pc4;
wire [31:0] next_pc;

wire [31:0] DataA;
wire [31:0] DataB;

wire [31:0] Imm;
```

```
wire [31:0] alu_inA;

wire [31:0] alu_inB;

//pc register

pc_reg PC(



   .clk(clk),

   .reset(reset),



   .next_pc(next_pc),

   .pc(pc)



);

//pc+4 adder

pc_adder ADDER(



   .pc(pc),

   .pc4(pc4)



);

//instr memory

instruction_memory IMEM(



   .pc(pc),
```

```
   .inst(inst)



);

//imm generator

imm_generator IMM(



   .inst(inst),

   .ImmSel(ImmSel),

   .Imm(Imm)



);

//register file

register_file RF(



   .clk(clk),

   .RegWEn(RegWEn),



   .AddrA(inst[19:15]),

   .AddrB(inst[24:20]),



   .AddrD(inst[11:7]),



   .DataD(wb),
```

```
   .DataA(DataA),

   .DataB(DataB)



);

//comparator

branch_comparator COMP(



   .DataA(DataA),

   .DataB(DataB),



   .BrEq(BrEq),

   .BrLT(BrLT)



);

//a mux

a_mux AMUX(



   .DataA(DataA),

   .pc(pc),



   .ASel(ASel),
```

```
   .A(alu_inA)



);

//b mux

b_mux BMUX(



   .DataB(DataB),

   .Imm(Imm),



   .BSel(BSel),



   .B(alu_inB)



);

//alu mux

alu ALU(



   .A(alu_inA),

   .B(alu_inB),



   .ALUSel(ALUSel),



   .result(alu_result)
```

```
);
//write back
wb_mux WBMUX(

   .alu_result(alu_result),
   .pc4(pc4),

   .WBSel(WBSel),

   .wb(wb)

);
//pc mux
pc_mux PCMUX(

   .pc4(pc4),
   .target(alu_result),

   .PCSel(PCSel),

   .next_pc(next_pc)
```

```
);

endmodule

//testbench

module datapath_32bit_tb();



reg clk;

reg reset;



reg PCSel;

reg RegWEn;

reg ASel;

reg BSel;



reg [2:0] ALUSel;

reg [1:0] ImmSel;



reg WBSel;



wire [31:0] pc;

wire [31:0] inst;

wire [31:0] alu_result;

wire [31:0] wb;
```

```
wire BrEq;

wire BrLT;



datapath_32bit DUT(



   .clk(clk),

   .reset(reset),



   .PCSel(PCSel),

   .RegWEn(RegWEn),

   .ASel(ASel),

   .BSel(BSel),



   .ALUSel(ALUSel),

   .ImmSel(ImmSel),



   .WBSel(WBSel),



   .pc(pc),

   .inst(inst),

   .alu_result(alu_result),

   .wb(wb),
```

```
   .BrEq(BrEq),

   .BrLT(BrLT)



);



always #5 clk = ~clk;



initial

begin



   clk = 0;

   reset = 1;

   PCSel = 0;

   RegWEn = 0;

   ASel = 0;

   BSel = 0;

   ALUSel = 3'b000;

   ImmSel = 2'b00;

   WBSel = 0;

   #10;

   reset = 0;

   // ADDI x5,x0,7
```

```
    RegWEn = 1;

    ASel = 0;

    BSel = 1;

    ALUSel = 3'b000;

    ImmSel = 2'b00;

    #10;

    // ADDI x6,x0,5

    #10;

    // ADD x7,x5,x6



    BSel = 0;

    ALUSel = 3'b000;

    #10;

    // SUB x8,x5,x6



    ALUSel = 3'b001;

    #10;

    RegWEn = 0;

    #10;

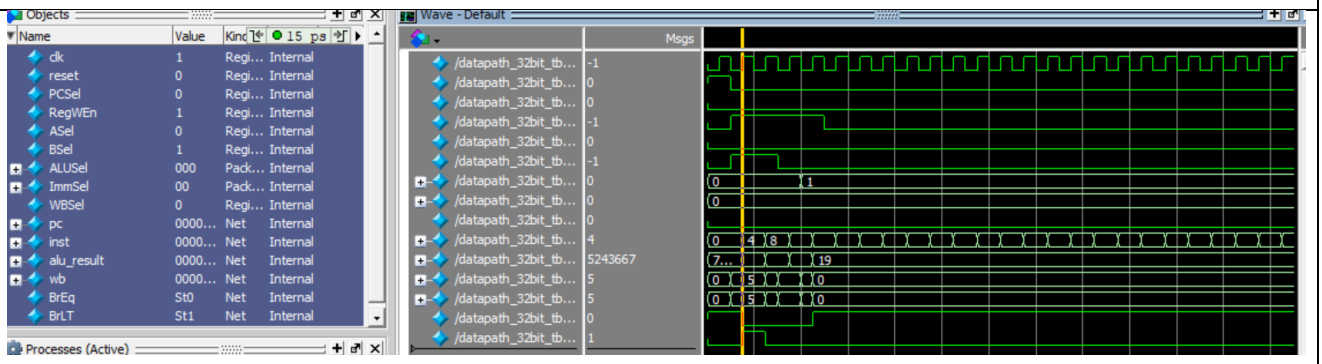


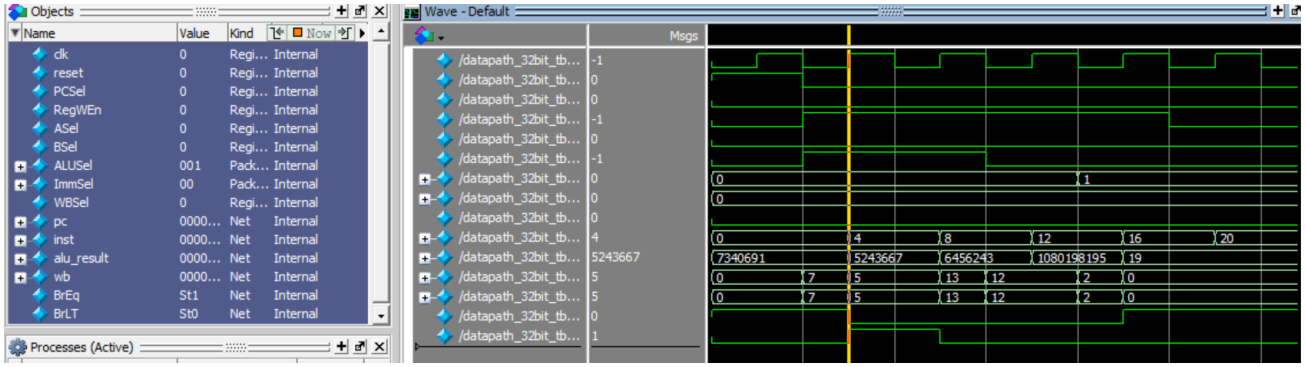
end



endmodule
```

**Output:**



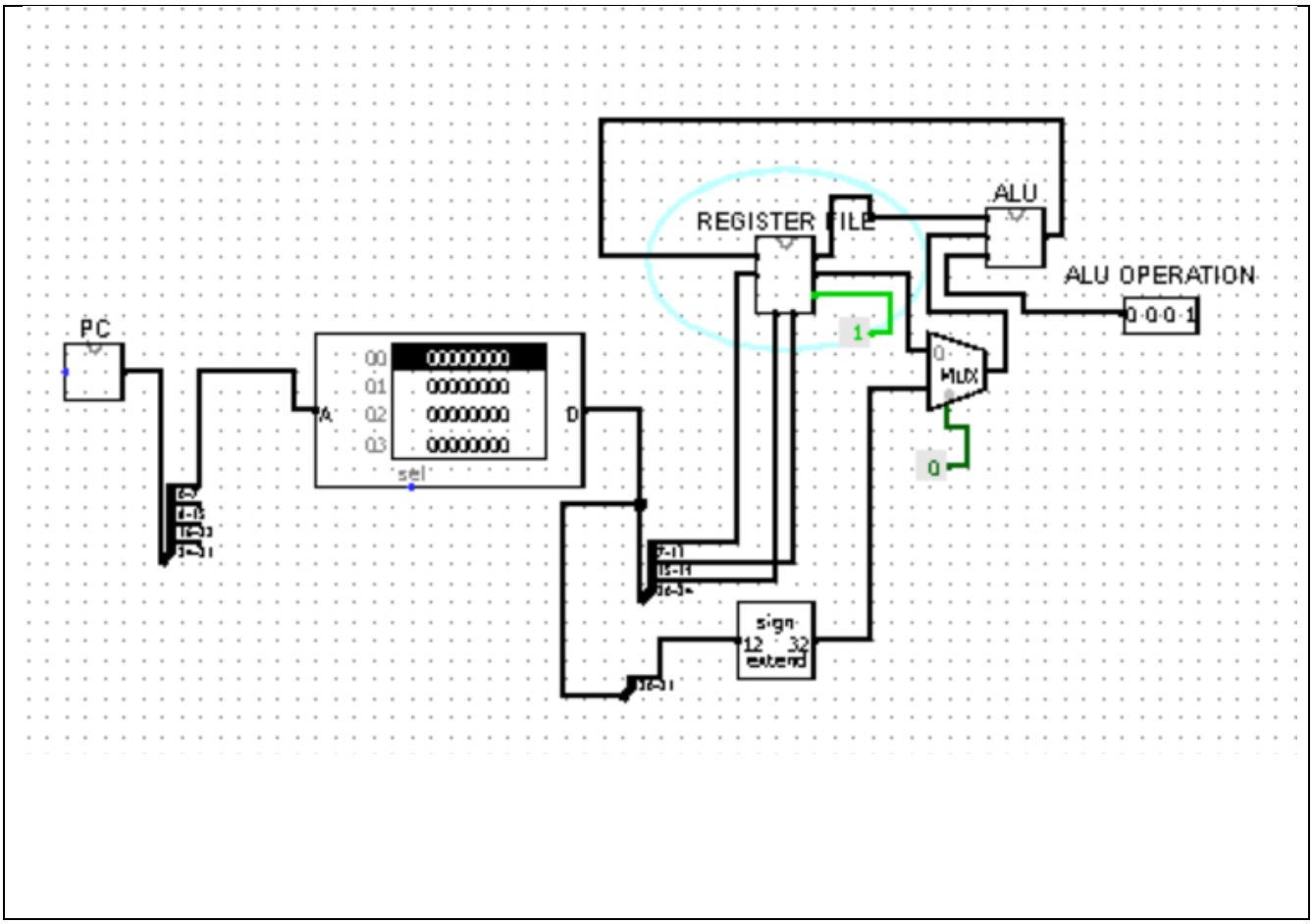




## Task 2:

**Description:**

Following is the logisim B y using the cubcircuits of ALU32BIT REISTER FILE 32 BIT and pc 32bit:

**Output:**


REGISTER FILE
ALU
ALU OPERATION
0 0 0 1
PC
00 00000000
01 00000000
02 00000000
03 00000000
A
D
sel
MUX
1
0
sign
12 32
extend